💾 TEMA 9: Base de Dados — Armazenamento Local e CRUD

Domine o funcionamento interno da persistência no navegador: o ciclo de vida do LocalStorage, a serialização JSON, a arquitetura CRUD completa e as regras essenciais para adicionar campos sem quebrar sistemas legados.

LocalStorage · JSON Stringify/Parse · Ciclo CRUD · Evolução de Esquema · Nullish Coalescing (??) · I/O Blob & FileReader

🗄️ 1. Memória Volátil (RAM) vs. Armazenamento Persistente

> 💡 **Conceito-chave:** quando declaramos variáveis em JavaScript (como `let alunos = []`), elas existem apenas na **memória RAM** do navegador. Se o usuário recarrega a página (F5) ou fecha o app, todos os dados somem. O **LocalStorage** resolve isso gravando os dados no disco do dispositivo sob o domínio da aplicação.

O LocalStorage é um banco de dados embutido do tipo **Chave-Valor (Key-Value)** presente em todos os navegadores modernos e WebViews móveis. Ele mantém as informações guardadas mesmo que o aparelho seja desligado.

🪜 As 3 Camadas da Persistência Web

CAMADA 1

🖥️ Interface (DOM)

Campos de digitação (`<input>`) e visualização em tabela (`<tbody>`). É a ponta visual com a qual o usuário interage.

CAMADA 2

⚡ Estado em RAM

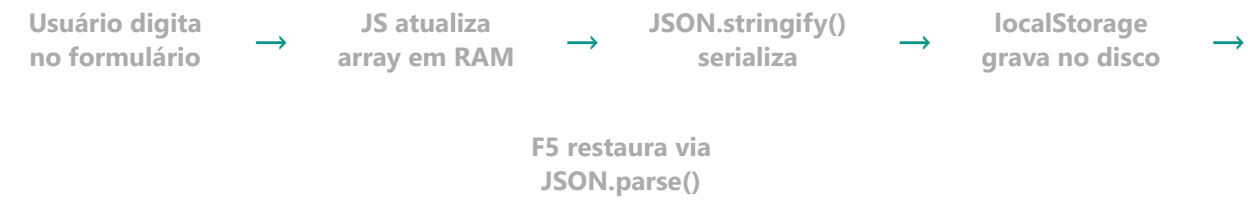
O array de objetos JavaScript (`alunos = [{...}]`). É rápido e mutável, mas totalmente volátil.

CAMADA 3

💾 Persistência (Disco)

O LocalStorage do navegador. Armazena a versão congelada em texto dos dados para restauração futura.

🔄 O Ciclo Vital de Persistência



🌉 2. A Ponte Serializadora: JSON.stringify e JSON.parse

> ⚠️ **A Regra de Ouro:** o LocalStorage **SÓ armazena Strings (texto puro)**! Se você tentar passar um array ou objeto direto para o `setItem()`, o JavaScript converterá para `"[object Object]"`, corrompendo irreversivelmente seus dados.

Para contornar essa restrição técnica, utilizamos as duas funções da biblioteca nativa `JSON`:

Método	Direção	O que faz na prática
`JSON.stringify(objeto)`	RAM ➔ Disco	Transforma o array/objeto estruturado em uma string de texto serializada pronta para o `setItem`.
`JSON.parse(texto)`	Disco ➔ RAM	Reconstrói a string textual do `getItem`, restaurando-a como array de objetos manipulável em memória.

💻 As duas funções centrais de sincronia

```
// 1. PERSISTIR: Salva a RAM no disco
function persistir() {
    localStorage.setItem(CHAVE, JSON.stringify(alunos));
}

// 2. CARREGAR: Restaura o disco para a RAM ao iniciar a página
function carregar() {
    const salvo = localStorage.getItem(CHAVE);
    if (salvo) {
        alunos = JSON.parse(salvo);
        // Recalcula o próximo ID incremental para evitar colisões
        proximoId = alunos.length ? Math.max(...alunos.map(a => a.id)) + 1 : 1;
    }
```

```
    listar();
}
```

⚙️ 3. A Arquitetura das 4 Operações (CRUD)

CRUD é o acrônimo universal da engenharia de software para as quatro operações fundamentais sobre qualquer base de dados:

C — CREATE

Criar / Inserir

Lê os valores dos inputs e insere um novo objeto no array com `alunos.push({ id, ... })`.

R — READ

Ler / Renderizar

Percorre o array com `alunos.forEach()` e gera dinamicamente as tags `<tr><td>` na tabela.

U — UPDATE

Atualizar / Editar

Localiza o registro com `alunos.find()` e substitui os dados existentes pelos novos valores digitados.

D — DELETE

Excluir / Deletar

Remove o elemento do array utilizando `alunos.filter(a => a.id !== idAlvo)`.

🎯 A Técnica do Ponteiro de Edição (idEditando)

Para reaproveitar o mesmo formulário e botão tanto para **cadastrar** quanto para **atualizar**, usamos uma variável de controle chamada `idEditando`:

```
let idEditando = null; // null = cadastrando novo | número = editando aluno exist

function salvar() {
    if (idEditando) {
        // [UPDATE] Se há um ID ativo, atualiza o aluno existente em vez de criar
        const a = alunos.find(x => x.id === idEditando);
        a.nome = nome.value;
        a.email = email.value;
        idEditando = null; // Libera o modo de edição
    } else {
        // [CREATE] Se não há ID, adiciona como novo registro com ID sequencial
        alunos.push({ id: proximoId++, nome: nome.value, email: email.value });
    }
    nome.value = email.value = ""; // Limpa os campos
    persistir();
```

```
        listar();
    }
```

🧩 4. Evolução de Esquema: Adicionando Novos Campos

> 💡 **O Desafio do Desenvolvedor:** sistemas reais mudam com o tempo. Adicionar um novo campo (como `idade`, `telefone` ou `curso`) exige uma intervenção cirúrgica e coordenada em 5 pontos exatos da aplicação, sem quebrar o que já existe.

📋 O Checklist Obrigatório de Manutenção

Local	O que deve ser feito	Exemplo Prático (Campo Idade)
1. HTML Form	Adicionar o novo `<input>` para captura	`<input id="idade" type="number" placeholder="Idade">`
2. HTML Tabela	Adicionar a nova coluna no `<thead>`	`<tr><th>ID</th>...<th>Idade</th><th>Ações</th></tr>`
3. JS: Create	Incluir a propriedade no objeto e na limpeza	`alunos.push({ ..., idade: idade.value });` e `idade.value = "";`
4. JS: Read	Renderizar a célula `<td>` com proteção nula	`<td>${a.idade ?? "-"}</td>`
5. JS: Update	Preencher no `editar()` e atualizar no `salvar()`	`idade.value = a.idade || "";` e `a.idade = idade.value;`

> 🛡️ **O Operador Nullish Coalescing (`??`):** se você cadastrou alunos no passado e agora adicionou a coluna idade, esses alunos antigos têm `a.idade === undefined`. Se você escrever apenas `${a.idade}`, a tabela exibirá feiamente o texto "*undefined*". Usar `${a.idade ?? "-"}` faz o JavaScript exibir um traço caso o valor seja nulo ou indefinido!

📁 5. Entrada e Saída: Backup Físico com Blob e FileReader

O LocalStorage fica preso dentro do navegador daquele computador específico. Para transferir dados entre máquinas ou criar cópias de segurança (backups), implementamos rotinas de

exportação e importação física de arquivos `.json`:

📤 Exportação de Arquivos com Blob

```
function exportar() {
    // Converte o array da RAM em texto identado legível
    const texto = JSON.stringify(alunos, null, 2);
    // Cria um arquivo binário em memória virtual (Blob)
    const blob = new Blob([texto], { type: "application/json" });
    // Cria um link temporário invisível e força o download
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "alunos.json";
    document.body.appendChild(a);
    a.click();
    a.remove();
    URL.revokeObjectURL(a.href);
}
```

📥 Importação Segura com FileReader e Validação

```
function importar(evento) {
    const arquivo = evento.target.files[0];
    if (!arquivo) return;

    const leitor = new FileReader();
    leitor.onload = () => {
        try {
            const dados = JSON.parse(leitor.result);
            // Defesa de integridade: checa se o conteúdo é de fato uma lista
            if (!Array.isArray(dados)) throw new Error("O arquivo não contém uma

            alunos = dados;
            proximoId = alunos.length ? Math.max(...alunos.map(a => a.id)) + 1 :

            persistir();
            listar();
            alert("Base de dados importada com sucesso!");
        } catch (e) {
            alert("Arquivo corrompido ou inválido: " + e.message);
        }
    };
    leitor.readAsText(arquivo);
}
```

> 💡 **Vantagem Estrutural:** repare que as funções `persistir()`, `exportar()` e `importar()` **não precisam ser alteradas** quando você adiciona novos campos (como

> idade). O formato JSON serializa e restaura o objeto integralmente, qualquer que seja a quantidade de propriedades!

📊 6. Tabela Resumo das Operações de Base de Dados

Operação	Comando Principal	Alvo	O que acontece se esquecer?
Gravar no Disco	`localStorage.setItem(CHAVE, str)`	LocalStorage	Dados somem ao recarregar a página (F5).
Ler do Disco	`localStorage.getItem(CHAVE)`	LocalStorage	A aplicação sempre iniciará vazia.
Serializar	`JSON.stringify(array)`	RAM ➔ String	O LocalStorage grava `"[object Object]"` e corrompe.
Desserializar	`JSON.parse(string)`	String ➔ RAM	O array vira texto puro e não aceita métodos como `.map()` e `.push()`.
Limpar Total	`localStorage.removeItem(CHAVE)`	LocalStorage	A chave continua salva ocupando espaço e memória.

📝 7. Questionário de Fixação (15 Questões)

Responda às questões e clique em "Verificar Respostas" para avaliar seu domínio sobre armazenamento local e manipulação de bases de dados:

1 **O que acontece com os dados armazenados apenas em variáveis JavaScript (como `let alunos = []`) quando o usuário atualiza a página (F5)?**

- Os dados são salvos automaticamente no disco rígido
- **Os dados são perdidos imediatamente, pois a memória RAM é volátil**
- Os dados vão para a nuvem do Google
- Os dados ficam congelados na tag <body>

2 **Qual é a restrição de tipo de dado fundamental imposta pelo LocalStorage do navegador?**

- Ele armazena apenas números inteiros
- **Ele aceita apenas texto puro (Strings)**
- Ele só guarda imagens em formato PNG
- Ele armazena objetos nativos binários diretamente

3 **Para que serve o método `JSON.stringify()` no processo de persistência?**

- **Converter um array/objeto JavaScript em uma string de texto serializada**
- Apagar todas as chaves do LocalStorage
- Abrir a janela de impressão do navegador
- Traduzir os campos do formulário para inglês

4 **O que faz o método `JSON.parse()` ao inicializar a aplicação com dados salvos?**

- Gera um arquivo PDF para download
- **Reconstrói a string textual armazenada transformando-a em objeto/array JavaScript em memória**

- Ordena os alunos em ordem alfabética automaticamente
- Valida se o endereço de e-mail possui @

5 **Qual é o papel da variável `idEditando` no fluxo de operações CRUD?**

- Bloquear o acesso de usuários não cadastrados
- **Atuar como bandeira (flag) para indicar se o botão 'Salvar' deve criar um novo aluno ou atualizar um existente**
- Contar quantas vezes a página foi recarregada
- Limpar a memória RAM do computador

6 **Ao carregar os alunos salvos, como o script garante que o próximo ID inserido não colidirá com IDs já existentes?**

- Sorteia um número aleatório entre 1 e 1000
- **Calcula o maior ID existente com `Math.max(...)` e soma 1**
- Zera todos os IDs anteriores a cada inicialização
- Usa sempre o número de caracteres do nome

7 **Ao acrescentar um campo 'Idade' ao formulário, qual alteração deve ser feita no cabeçalho da tabela HTML para não desalinhá-la?**

- **Adicionar uma nova tag `<th>Idade</th>` na mesma ordem em que os dados aparecem no corpo**
- Excluir a tag <thead> inteira
- Mudar a propriedade border-collapse para separate

- Criar uma segunda tabela separada apenas para idades

8 **Qual operador é a melhor prática para exibir um traço ("-") quando um aluno antigo não possui a propriedade idade cadastrada?**

- **O operador de coalescência nula:** `${a.idade ?? "-"}`
- O operador de soma: `a.idade + "-"`
- A função `delete a.idade`
- O comando `window.alert(a.idade)`

9 **Qual comando remove um registro específico do array em memória utilizando seu identificador exclusivo?**

- `alunos.pop()`
- **`alunos = alunos.filter(a => a.id !== id);`**
- `alunos.slice(0, 1)`
- `localStorage.clear()`

10 **Na rotina de exportação para download físico, para que serve a instrução** `new Blob([texto], { type: "application/json" })` **?**

- Para criptografar os dados com senha bancária
- **Para criar um arquivo binário em memória virtual que o navegador possa baixar como arquivo real**
- Para compactar a tabela no formato ZIP
- Para enviar os alunos automaticamente para o servidor remoto

11 **Ao importar um arquivo JSON externo com `FileReader`, por que envolvemos a leitura em um bloco `try...catch`?**

- Para acelerar a velocidade de processamento da CPU
- **Para capturar erros de sintaxe ou arquivos corrompidos sem travar a aplicação**
- Porque o JavaScript exige try/catch em todas as linhas
- Para impedir que arquivos com mais de 10 linhas sejam lidos

12 **O que acontece com a função `persistir()` quando você adiciona novos atributos (como telefone ou idade) aos alunos?**

- Ela precisa ser completamente reescrita
- **Ela não precisa de nenhuma alteração, pois o JSON serializa dinamicamente qualquer propriedade existente**
- Ela falha e passa a salvar apenas strings vazias
- Exige a criação de uma nova chave no LocalStorage para cada novo campo

13 **O LocalStorage apaga todos os seus dados automaticamente assim que o usuário fecha a aba do navegador.**

- ✅ Verdadeiro
- **❌ Falso**

14 **A instrução `localStorage.removeItem(CHAVE)` remove a chave selecionada sem apagar outras chaves de outros sistemas guardadas no mesmo navegador.**

- **✅ Verdadeiro**
- ❌ Falso

15 **O comando encadeado** `nome.value = email.value = idade.value = "";` **limpa os três campos de entrada de forma eficiente após o salvamento.**

- (●) ✅ **Verdadeiro**
- ( ) ❌ Falso

✅ Verificar Respostas 🔄 Limpar Respostas

📊 Resultado da Avaliação

15 / 15

Aproveitamento: 100%